IF2110 Algoritma dan Pemrograman 2

Pohon Biner sebagai Struktur Data Rekursif

Tim Pengajar IF2110

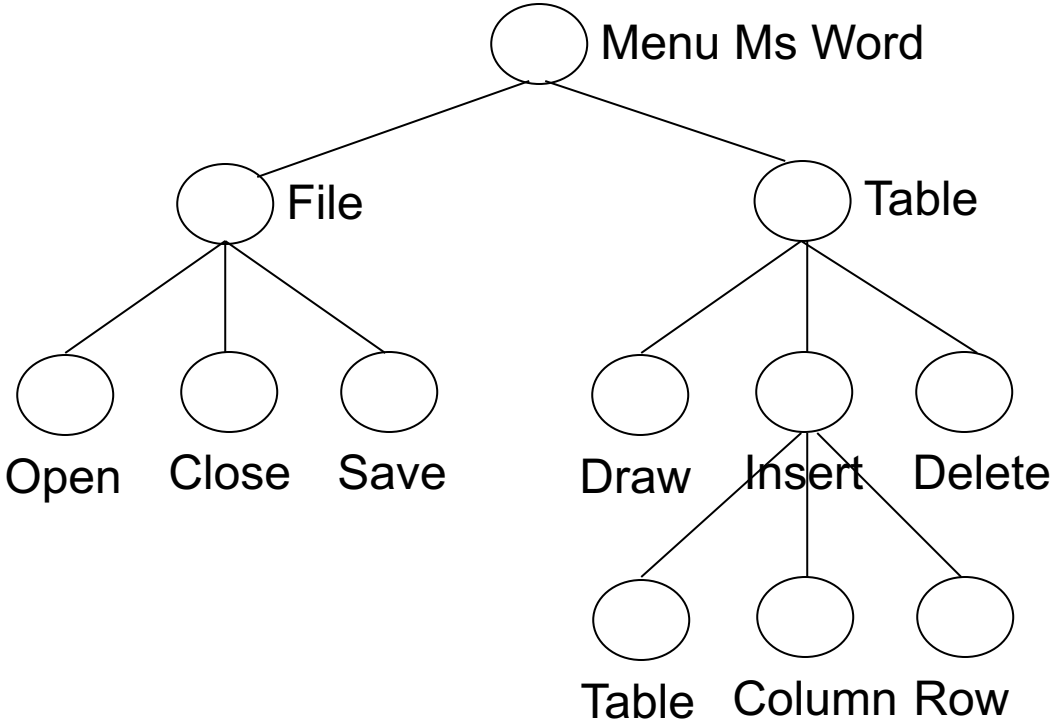
Sekolah Teknik Elektro dan Informatika

Tujuan

- Mahasiswa memahami definisi pohon dan pohon biner
- Berdasarkan pemahaman tersebut, mampu membuat fungsi sederhana yang memanipulasi pohon
- Mahasiswa mampu mengimplementasi fungsi pemroses pohon dalam LISP → melalui praktikum

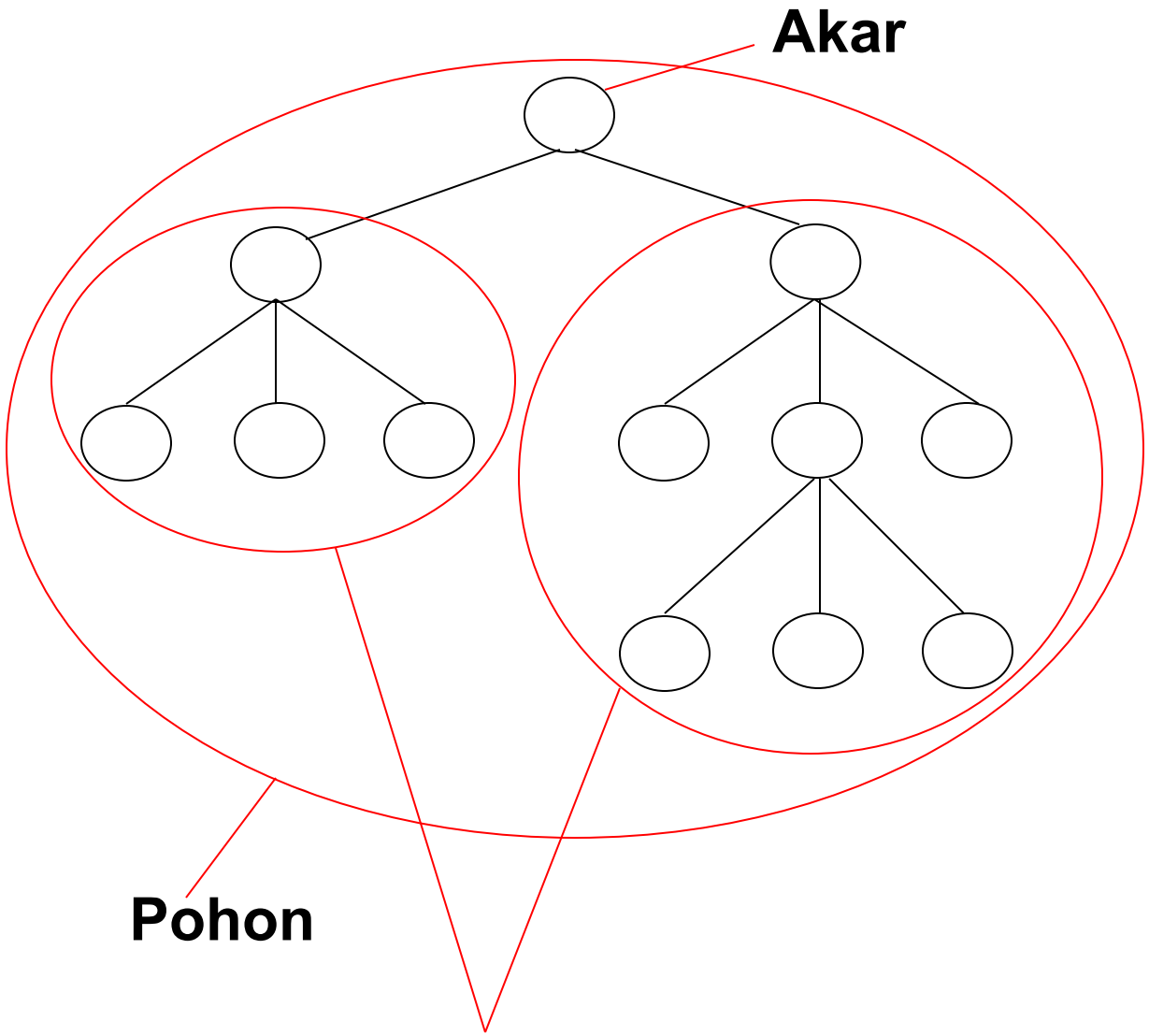
Contoh Persoalan Direpresentasi Sbg Pohon

- Menu dalam Aplikasi Komputer
- Contoh (Ms Word):
  - File
    - Open
    - Close
    - Save
  - Table
    - Draw
    - Insert
      - Table
      - Column
      - Row
    - Delete



Pohon

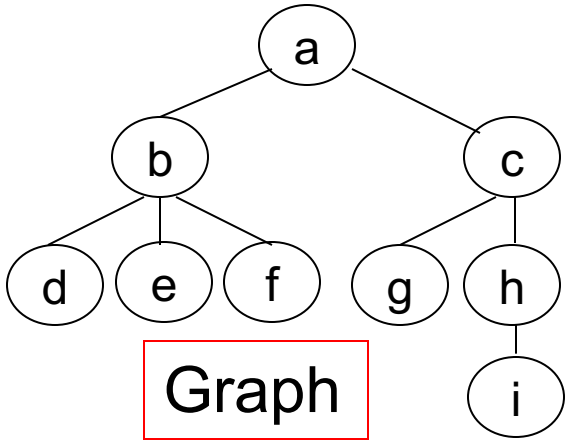
Akar



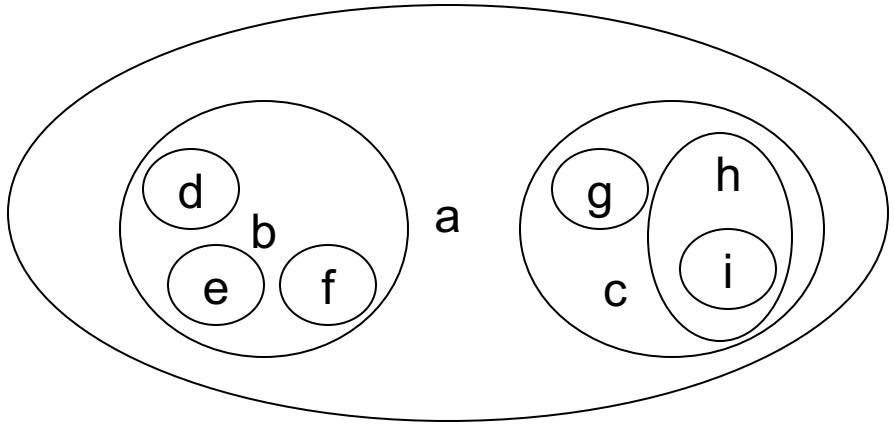
SubPohon

Elemen Pohon:
-Akar → basis
-Sub Pohon (sub himpunan yang berupa pohon) → rekurens

Beberapa Ilustrasi Representasi



Graph



Himpunan

Indentasi

```
a
    b
        d
        e
        f
    c
        g
        h
            i
```

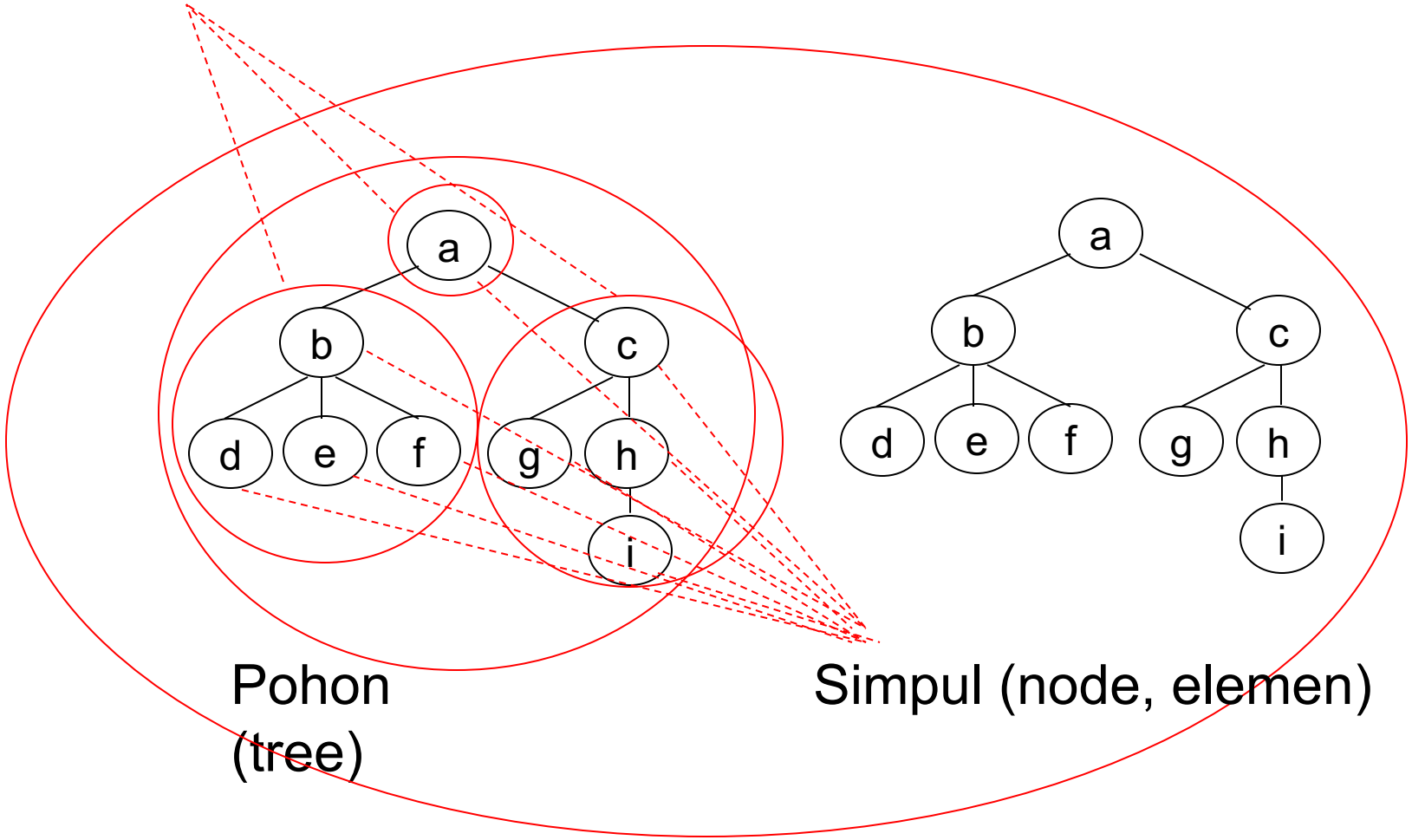
Bentuk Linier

Prefix: (a,((b,((d,()),(e,()),(f,()))),(c,((g,()),(h,((i,()))))))) *atau*
(a,(b,(d),(e),(g)),(c,(g),(h,(i))))

Postfix: (((d),(e),(f),b),((g),((i),h),c),a)

Istilah

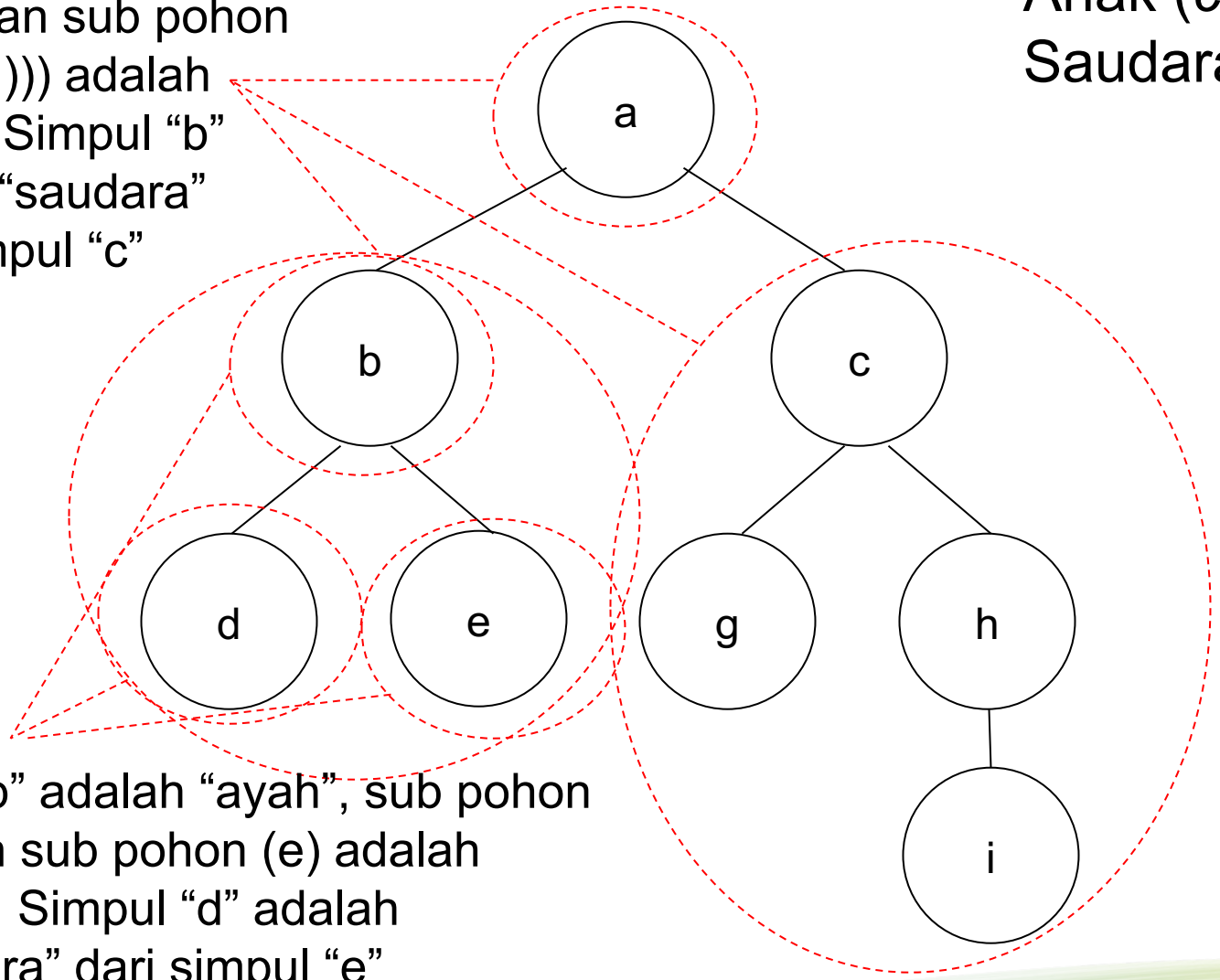
Cabang → akar “a” memiliki cabang 2 sub pohon yaitu (b (d e f)) dan (c (g h(i)))



Istilah

Akar “a” adalah “ayah”, sub pohon (b (d e)) dan sub pohon (c (g h(i))) adalah “anak”. Simpul “b” adalah “saudara” dari simpul “c”

Ayah (*father*)
Anak (*child*)
Saudara (*sibling*)

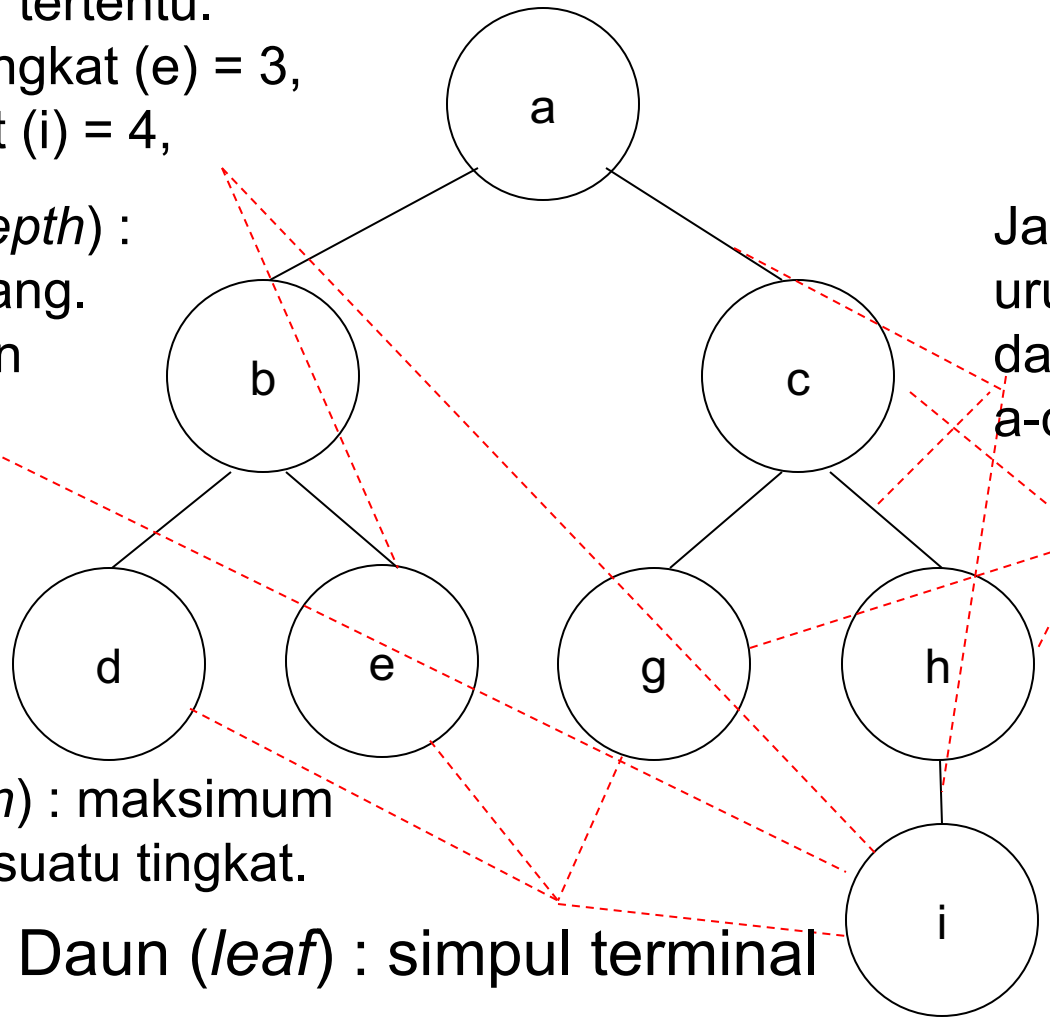


Akar “b” adalah “ayah”, sub pohon (d) dan sub pohon (e) adalah “anak”. Simpul “d” adalah “saudara” dari simpul “e”

Istilah

Tingkat (*level*) : panjang jalan dari akar sampai simpul tertentu. Cth: tingkat (e) = 3, tingkat (i) = 4,

Kedalaman (*depth*) : tingkat terpanjang. Cth: kedalaman pohon=4



Jalan (*path*) : urutan tertentu dari cabang, cth: a-c-h-i

Derajat : banyaknya anak sebuah simpul. Cth, derajat(c)=2, derajat(h)=1, derajat(g)=0

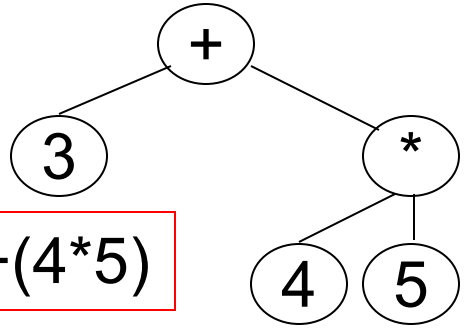
Lebar (*breadth*) : maksimum jml simpul pd suatu tingkat.

Daun (*leaf*) : simpul terminal

Pohon Biner

- Definisi
  1. Mungkin kosong
  2. Ada simpul akar dan dua anak yang berupa pohon biner, satu sub pohon kiri dan satu sub pohon kanan. Anak mungkin kosong (kembali ke definisi 1).

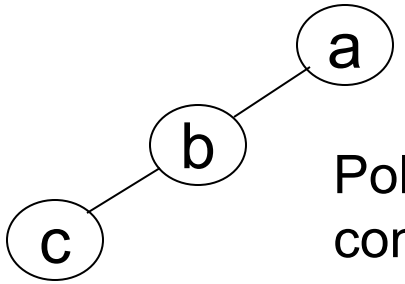
Penulisan sub pohon:
dgn ( )



3+(4*5)

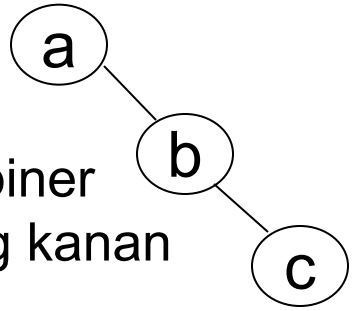
(+ (3 ( ) ( )) (* (4 ( )( )) (5 ( )( ))))
atau (+ (3) (* (4)(5)))

Pohon condong/*skewed tree*



Pohon biner condong kiri

Notasi Prefix: (a (b (c ( ) ( )) ( )) ( ))
atau (a (b (c)( )) ( ))



Pohon biner condong kanan

Notasi Prefix: (a ( ) (b ( )(c ( ) ( )))) atau
(a ( ) (b ( )(c)))

TYPE POHON BINER

DEFINISI DAN SPESIFIKASI TYPE

type Elemen: {tergantung type node}
type PohonBiner:<L: PohonBiner, A: Elemen, R: PohonBiner> {infix}, **atau**
type PohonBiner:<A: Elemen, L: PohonBiner, R: PohonBiner> {prefix}, **atau**
type PohonBiner:<L: PohonBiner, R: PohonBiner , A: Elemen> {postfix}
{Pohon Biner terdiri dari Akar yang berupa elemen, L dan R adalah Pohon Biner yang merupakan subpohon kiri dan subpohon kanan}

DEFINISI DAN SPESIFIKASI KONSTRUKTOR

{Perhatikanlah bhw konstruktor pohon biner dg basis pohon kosong ditulis:

a. Infix: //L A R\\
b. Prefix: //A L R\\
c. Postfix: //L R A\\}

Selektor

Akar: pohon biner tidak kosong → elemen

{ Akar(P) adalah akar dari P.
Jika P adalah //L A R\\ maka akar(P) = A}

Left: pohon biner tidak kosong → pohon biner

{ Left(P) adalah sub pohon kiri dari P.
Jika P adalah //L A R\\ maka left(P) = L}

Right: pohon biner tidak kosong → pohon biner

{ Right(P) adalah sub pohon kanan dari P.
Jika P adalah //L A R\\ maka Right(P) = R}

Predikat

IsTreeEmpty: pohon biner → boolean

{IsTreeEmpty(P) true jika P adalah // \\}

IsOneElmt: pohon biner → boolean

{IsOneElmt(P) true jika P adalah //A\\}

IsUnerLeft: pohon biner → boolean

*{IsUnerLeft(P) true jika P **hanya** mengandung sub pohon kiri, P adalah //L A\\}*

IsUnerRight: pohon biner → boolean

*{IsUnerRight(P) true jika P **hanya** mengandung sub pohon kanan, P adalah //A R\\}*

Predikat

IsBiner: pohon biner tidak kosong → <u>boolean</u>

{IsBiner(P) true jika P mengandung sub pohon kiri dan sub pohon kanan, P adalah // L A R \\}

IsExistLeft: pohon biner tidak kosong → <u>boolean</u>

{IsExistLeft(P) true jika P mengandung sub pohon kiri}

IsExistRight: pohon biner tidak kosong → <u>boolean</u>

{IsExistRight(P) true jika P mengandung sub pohon kanan}

Pohon Basis-0

- Definisi rekursif
  - Basis: pohon biner kosong adalah pohon biner {menggunakan predikat IsTreeEmpty}
  - Rekurens: Pohon biner tidak kosong terdiri dari sebuah simpul akar dan dua anak: sub pohon kiri dan sub pohon kanan. Sub pohon kiri dan sub pohon kanan adalah pohon biner

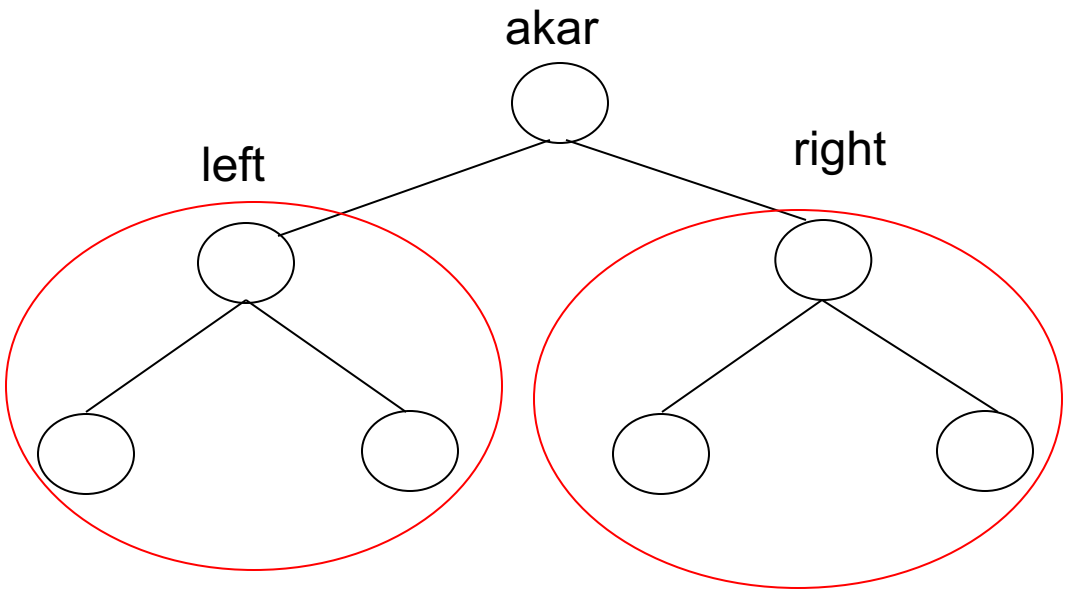
Pohon Basis-1

- Definisi rekursif
  - Basis: pohon biner yang hanya terdiri dari akar {menggunakan predikat IsOneElmt}
  - Rekurens: Pohon biner tidak kosong terdiri dari sebuah simpul akar dan dua anak yang salah satunya pasti tidak kosong: sub pohon kiri dan sub pohon kanan.
    - Gunakan IsUnerLeft, IsUnerRight, IsBiner untuk memastikan tidak terjadi pemrosesan pada pohon kosong

Menghitung Jumlah Elemen

NbElmt: PohonBiner → integer >= 0
{NbElmt(P) memberikan banyaknya elemen dari pohon P}

Berapa jumlah elemen pohon dilihat dari elemen current?



Jumlah elemen =
1 (utk akar) + Jumlah_Elemen (pohon kiri) + Jumlah _Elemen (pohon kanan)

Menghitung Jumlah Elemen, NbElmt

- Rekursif
  - Basis 0: jika pohon kosong maka jumlah elemen adalah 0
  - Rekurens: jumlah elemen = 1 (current element) + jumlah elemen pohon kiri + jumlah elemen pohon kanan

```
NbElmt (P):
  if IsTreeEmpty(P) then 0  {basis 0}
  else  {rekurens}
     NbElmt(Left(P)) + 1 + NbElmt(Right(P))
```

Menghitung Jumlah Elemen, NbElmt

- Basis 0

```
NbElmt (P):
  if IsTreeEmpty(P) then 0  {basis 0}
  else  {rekurens}
     NbElmt(Left(P)) + 1 + NbElmt(Right(P))
```

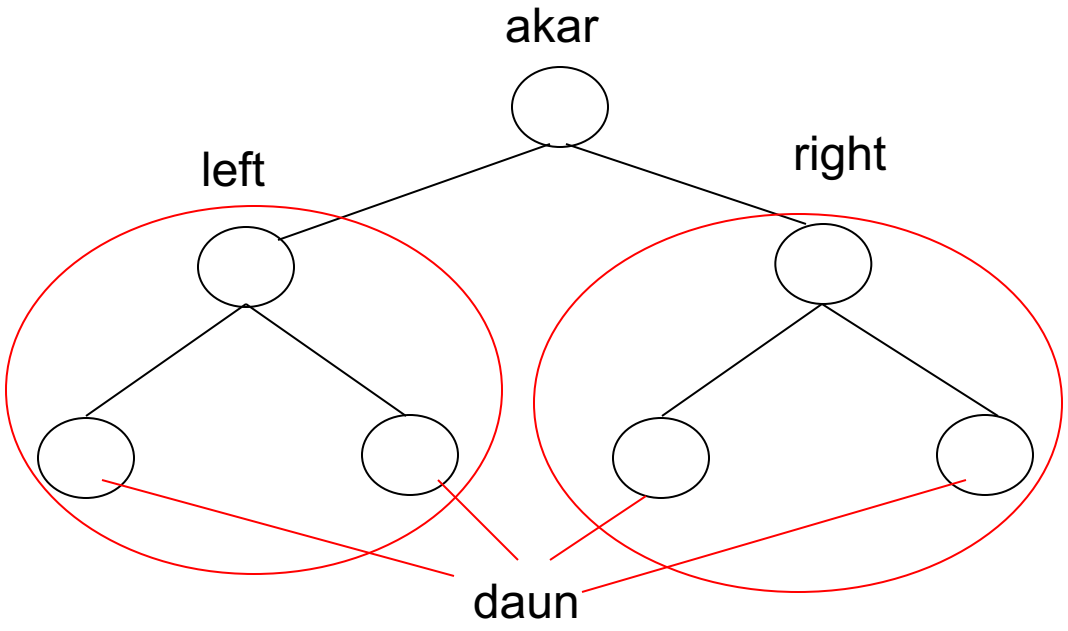
- Basis 1

```
NbElmt (P):
  if IsOneElmt(P) then 1  {basis 1}
  else  {rekurens}
     depend on P
        IsBiner(P):  NbElmt(Left(P)) + 1 + NbElmt(Right(P))
        IsUnerLeft(P):  NbElmt(Left(P)) + 1
        IsUnerRight(P):  1 + NbElmt(Right(P))
```

Menghitung Jumlah Daun

NbDaun: PohonBiner → integer >= 0
{NbDaun(P) memberikan banyaknya daun dari pohon P}

Berapa jumlah daun pohon dilihat dari elemen current?



Jumlah daun =
0 (utk akar) + Jumlah daun (pohon kiri) + Jumlah daun (pohon kanan)

Menghitung Jumlah Daun, NbDaun

- Menghitung daun pada dasarnya persoalan basis 1
- Analisis Kasus (pohon kosong adalah kasus khusus)
  - Pohon kosong : jumlah daun = 0
  - Pohon tidak kosong: jumlah daun dihitung dengan fungsi menghitung jumlah daun dengan basis-1

```
NbDaun (P):
  if IsTreeEmpty(P) then 0  { analisis kasus }
  else
     NbDaun1(P)
```

Menghitung Jumlah Daun, NbDaun

- Rekursif: Basis 1

```
NbDaun1 (P):
  if IsOneElmt(P) then 1  {basis 1}
  else  {rekurens}
     depend on P
        IsBiner(P):  NbDaun1(Left(P)) + NbDaun1(Right(P))
        IsUnerLeft(P):  NbDaun1(Left(P))
        IsUnerRight(P):  NbDaun1(Right(P))
```

Pohon Biner Pada Haskell

Tipe data BinTree dan Konstruktor

-- deklarasi tipe data, menggunakan format prefix

data **BinTree a** = Empty | Node a (BinTree a) (BinTree a) deriving (Show)

-- elemen BinTree dideklarasikan generik

-- konstruktor, untuk BinTree dengan elemen integer

makeBinTree :: Int -> (BinTree Int) -> (BinTree Int) -> (BinTree Int)

makeBinTree a Empty = Node a Empty Empty -- pohon 1 elemen

makeBinTree a l r = Node a l r

Contoh konstruksi sebuah BinTree

```
makeBinTree
        3
        (makeBinTree
                15
                Empty
                (makeBinTree 25 Empty Empty))
        (makeBinTree
                10
                (makeBinTree 50 Empty Empty)
                (makeBinTree 30 Empty Empty))
```

Selektor

akar :: (BinTree Int) -> Int
{- akar p adalah akar dari p. Jika p adalah //L A R\\ maka akar p = A -}
akar (Node a l r) = a

left :: (BinTree Int) -> (BinTree Int)
{- left p adalah sub pohon kiri dari p. Jika p adalah //L A R\\ maka left p = L -}
left (Node a l r) = l

right :: (BinTree Int) -> (BinTree Int)
{- right p adalah sub pohon kanan dari p. Jika p adalah //L A R\\ maka right p = R -}
right (Node a l r) = r

Predikat (1)

```
isTreeEmpty :: (BinTree Int) -> Bool
{- isTreeEmpty p true jika p adalah // \\ -}
isTreeEmpty Empty = True
isTreeEmpty (Node a l r) = False


isOneElmt :: (BinTree Int) -> Bool
{- isOneElmt p true jika p adalah //A\\ -}
isOneElmt p =
    not (isTreeEmpty p) &&
    isTreeEmpty (left p) && isTreeEmpty (right p)
```

Predikat (2)

```
isUnerLeft :: (BinTree Int) -> Bool
{- isUnerLeft p true jika p hanya mengandung sub pohon kiri, P adalah //L A\\ -}
isUnerLeft p = not (isTreeEmpty p) && not (isTreeEmpty (left p)) && isTreeEmpty (right p)

isUnerRight :: (BinTree Int) -> Bool
{- isUnerRight p true jika p hanya mengandung sub pohon kanan, P adalah //A L\\ -}
isUnerRight p = not (isTreeEmpty p) && isTreeEmpty (left p) && not (isTreeEmpty (right p))

isBiner :: (BinTree Int) -> Bool
{- isBiner p true jika p mengandung sub pohon kiri dan sub pohon kanan, p adalah // L A R \\ -}
isBiner p = not (isTreeEmpty p) && not (isTreeEmpty (left p)) && not (isTreeEmpty (right p))
```

Predikat (3)

```
isExistLeft :: (BinTree Int) -> Bool
{- isExistLeft p true jika p mengandung sub pohon kiri -}
isExistLeft p =
    not (isTreeEmpty p) && not (isTreeEmpty (left p))

isExistRight :: (BinTree Int) -> Bool
{- isExistRight p true jika p mengandung sub pohon kanan -}
isExistRight p =
    not (isTreeEmpty p) && not (isTreeEmpty (right p))
```

nbElmt

```
nbElmt :: (BinTree Int) -> Int
{- NbElmt p memberikan banyaknya elemen dari pohon p -}
nbElmt p =
  if isTreeEmpty p then   -- basis
    0
  else   -- rekurens
    nbElmt (left p) + 1 + nbElmt (right p)
```

nbDaun

```
nbDaun :: (BinTree Int) -> Int
{- nbDaun p memberikan banyaknya daun dari pohon p -}
nbDaun1 :: (BinTree Int) -> Int
{- nbDaun1 p memberikan banyaknya daun dari pohon p yang tidak kosong -}
nbDaun p =
        if isTreeEmpty p then 0 else nbDaun1 p
nbDaun1 p =
        if isOneElmt p then 1   -- basis
        else   -- rekurens
                if isBiner p then nbDaun1 (left p) + nbDaun1 (right p)
                else if isUnerLeft p then nbDaun1 (left p)
                        else nbDaun1 (right p)
```

Sumber

- Diktat “Dasar Pemrograman, Bag. Pemrograman Fungsional” oleh Inggriani Liem, revisi Februari 2014